Linear Models 1: Cats Lab

Nisia Trisconi and Dr. Matteo Tanadini

Applied Machine Learning and Predictive Modelling 1, FS26 (HSLU)

Contents

1 Getting data

```
d.cats <- read.csv("Cats.csv",
                   header = TRUE,
                   stringsAsFactors = TRUE)

##
str(d.cats)
```

```
'data.frame':   144 obs. of  3 variables:
 $ Sex         : Factor w/ 2 levels "F","M": 1 1 1 1 1 1 1 1 1 1 ...
 $ Body.weight : num  2 2 2 2.1 2.1 2.1 2.1 2.1 2.1 2.1 ...
 $ Heart.weight: num  7 7.4 9.5 7.2 7.3 7.6 8.1 8.2 8.3 8.5 ...
```

```
head(d.cats)
```

```
  Sex Body.weight Heart.weight
1   F         2.0          7.0
2   F         2.0          7.4
3   F         2.0          9.5
4   F         2.1          7.2
5   F         2.1          7.3
6   F         2.1          7.6
```

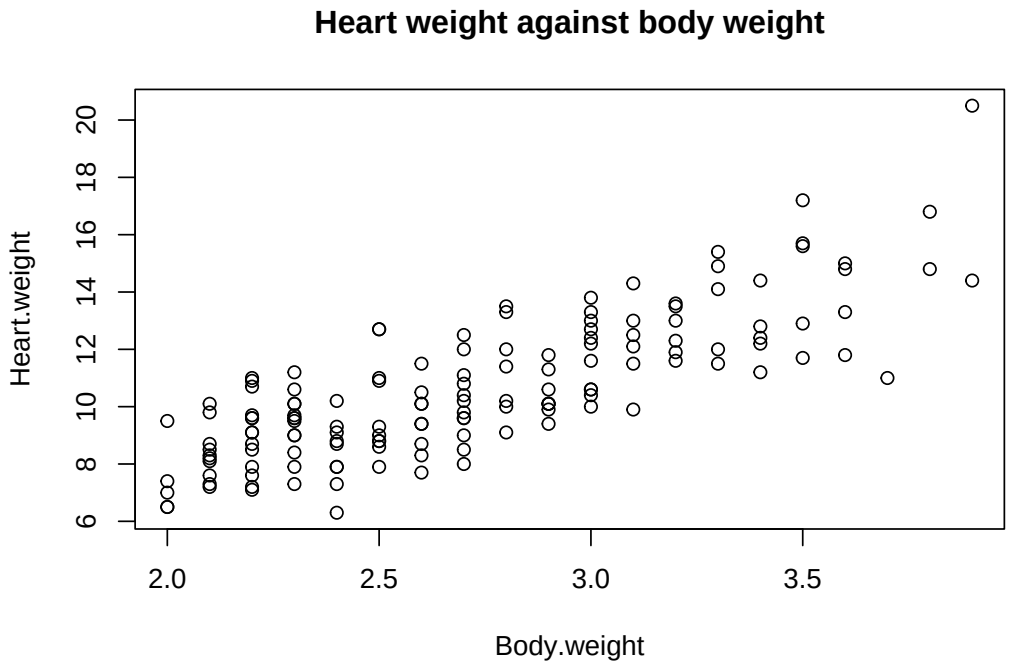
The data set has 144 observations and 3 variables. *Sex*, *Body.weight* and *Heart.weight*.

2 Graphical Analysis

The scope of this analysis is to be able to predict the heart weight based on sex and body weight.

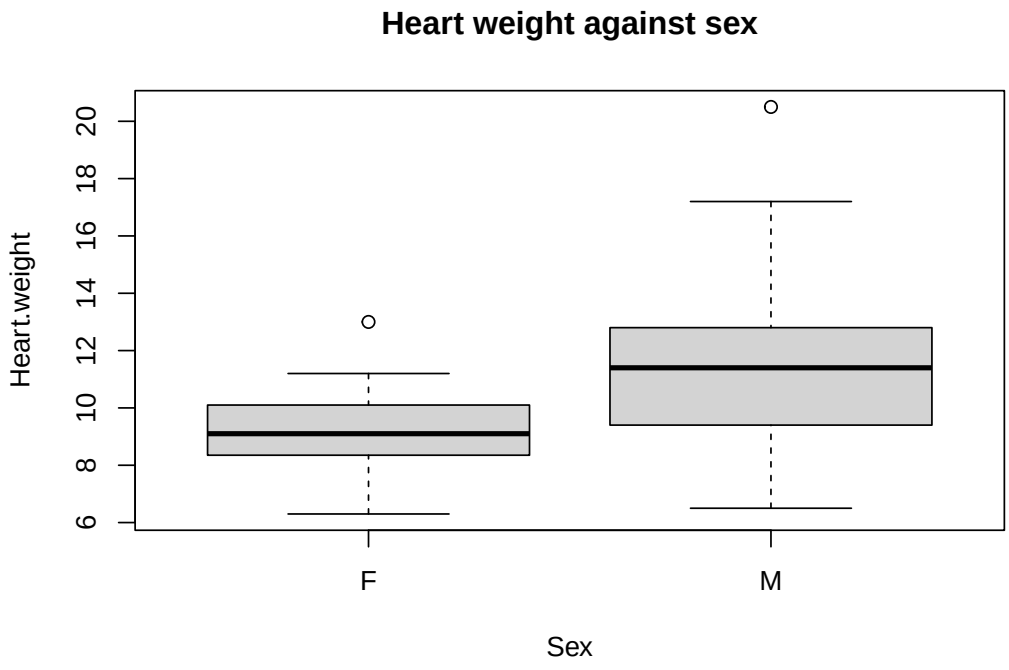
We start the graphical analysis by plotting the response variable (i.e., *Heart.weight*) against the predictor *Body.weight*. As *Body.weight* is a continuous variable, we create a scatterplot.

```
plot(Heart.weight ~ Body.weight, data = d.cats,
     main = "Heart weight against body weight")
```



There seems to be a positive relationship between these two variables. Let's inspect the predictor sex (*Sex*).

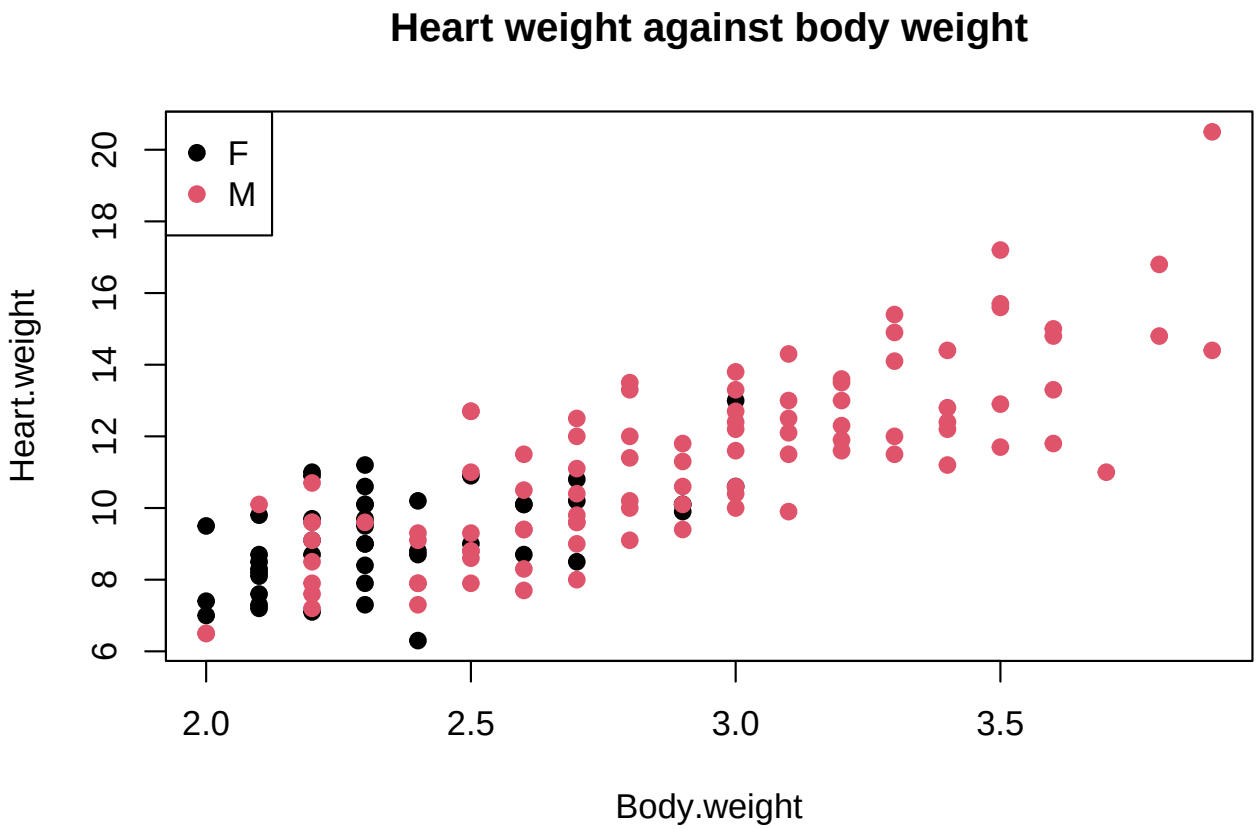
```
boxplot(Heart.weight ~ Sex, data = d.cats,
        main = "Heart weight against sex",
        ylab = "Heart.weight")
```



Male cats seem to have heavier hearts. Let's now consider both predictors in a single graph.

```
plot(Heart.weight ~ Body.weight, data = d.cats,
     col = Sex,
     pch = 19,
     main = "Heart weight against body weight")
##
```

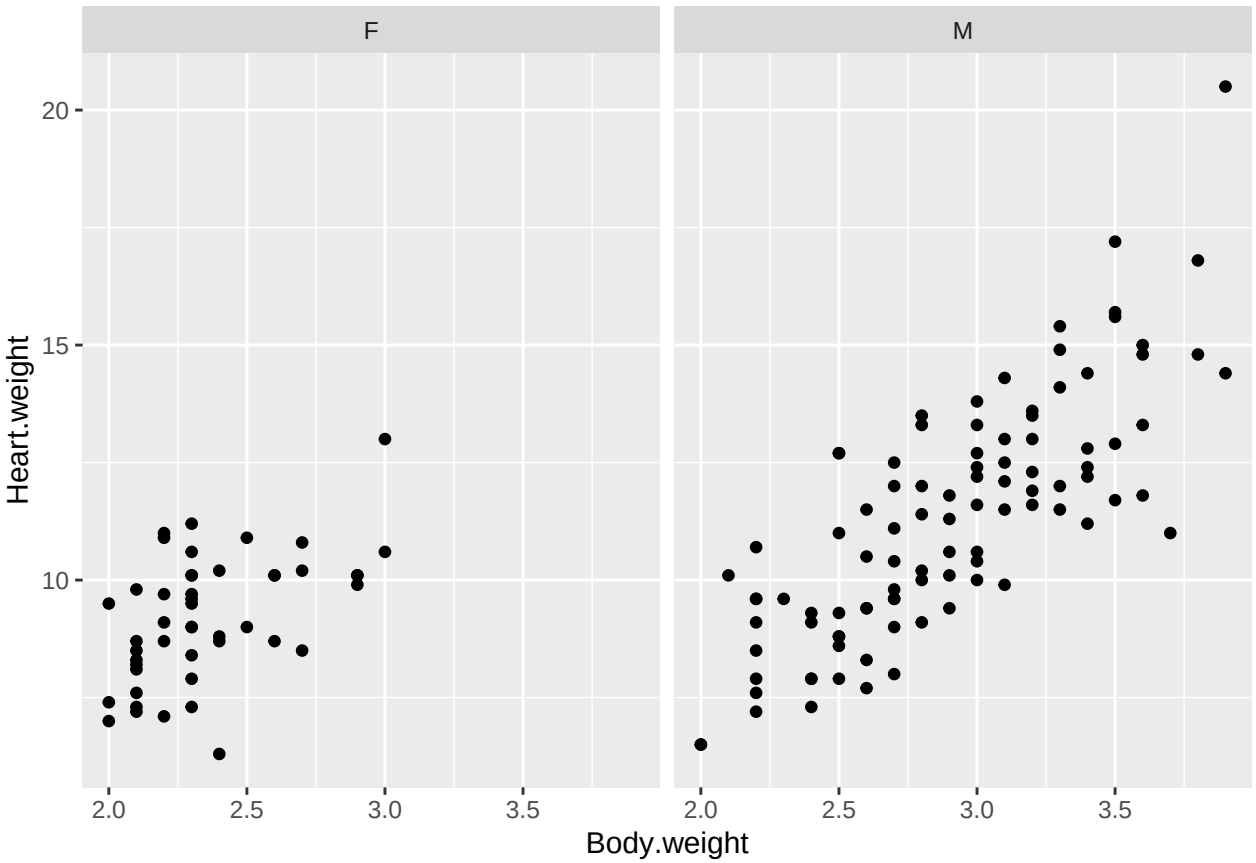
```
legend("topleft",
       pch = 19,
       legend = c("F","M"),
       col = c(1, 2))
```



Female cats seem to be more frequent on the left hand-side of this graph (i.e., they have lower body weights).

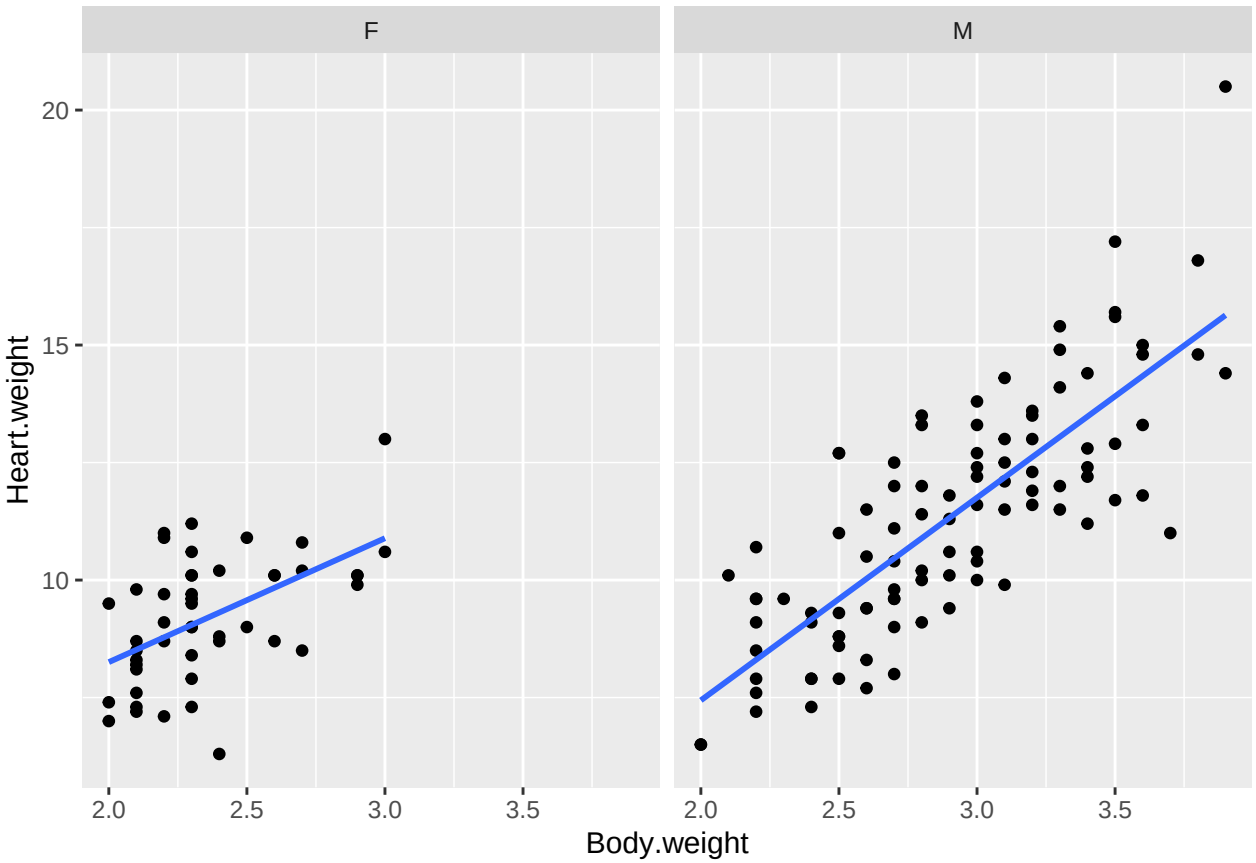
Unfortunately, some observations overlap. Therefore, we may want to use more sophisticated graphs to cope with this issue. To do that, we will use the add-on package `{ggplot2}`.

```
library(ggplot2)
ggplot(data = d.cats,
       mapping = aes(y = Heart.weight, x = Body.weight)) +
  geom_point() +
  facet_wrap(~ Sex)
```



The "facets" argument allows us to create a graph with two panels. This technique is known as "panelling" (Sarkar 2008). It is now simpler to gather information about each sex separately. This graph confirms that female cats have indeed lower body weights as well as lower heart weights. Nevertheless, for both sexes there seems to be a positive relationship between heart weight and body weight.

As we know that we are going to fit a linear model to this data, we may want to add a regression line in each panel. To produce the following graph we are going to use the `ggplot()` function that will be introduced later. For simplicity, the **R**-code to produce this graph is omitted.



Question: *what patterns do you observe by looking at this graph?*

3 Fitting models

3.1 Fitting a simple regression model

We start by fitting a simple regression model to this data set.

```
lm.cats <- lm(Heart.weight ~ Body.weight, data = d.cats)
```

In this model, only the continuous predictor *Body.weight* has been considered. Let's look at the summary information.

```
summary(lm.cats)
```

```
Call:
lm(formula = Heart.weight ~ Body.weight, data = d.cats)

Residuals:
   Min     1Q Median     3Q    Max
-3.569 -0.963 -0.092  1.043  5.124

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   -0.357      0.692   -0.52     0.61
Body.weight    4.034      0.250   16.12   <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.4 on 142 degrees of freedom
Multiple R-squared:  0.647, Adjusted R-squared:  0.644
F-statistic:  260 on 1 and 142 DF,  p-value: <2e-16
```

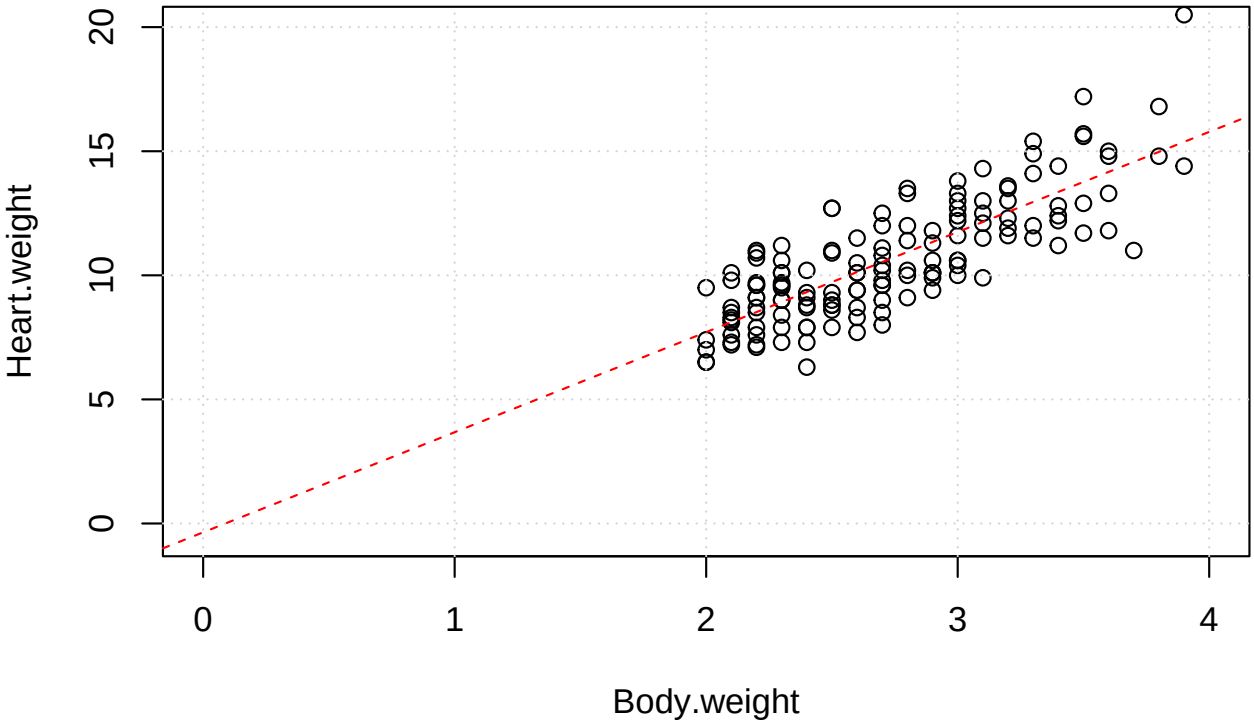
Let's look at the estimated regression coefficients.

```
coef(lm.cats)
```

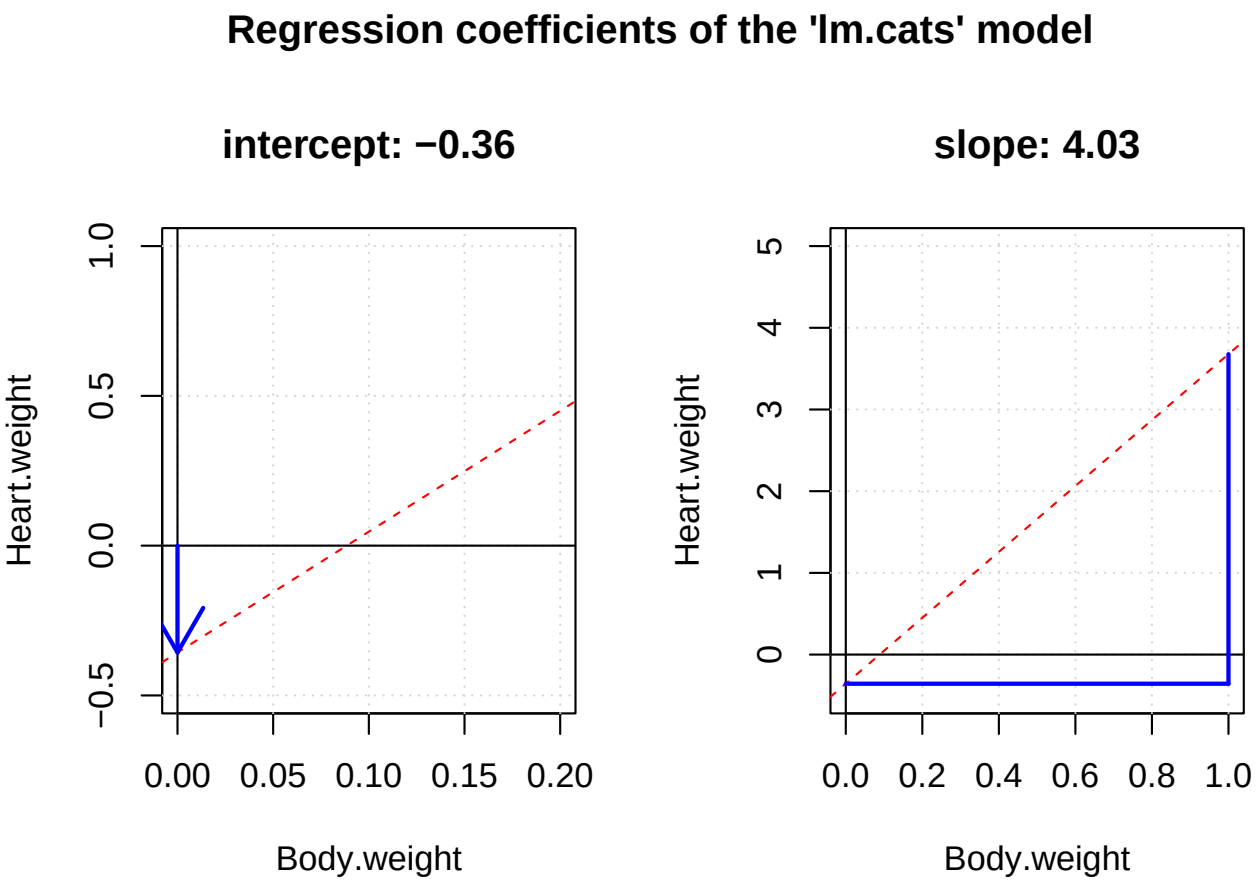
```
(Intercept) Body.weight
      -0.36        4.03
```

Question: *how do you interpret these two numbers? What is the "practical" meaning of the estimate for the intercept and for the slope? (2 minutes to discuss this with your colleagues)*

Let's visualise our model fit along with the raw data (**R**-code is omitted).



Let's zoom in on the areas of interest to better visualise the two regression coefficients (**R**-code is omitted).



The interpretation of the intercept is: "a cat of body weight zero has a heart weight of -0.36". Having a negative weight is obviously nonsensical. In the exercises we will see how to solve the issue of the interpretation of the intercept for this model[1].

On the other hand, the interpretation of the second coefficient (which is the slope for body weight) is: "for each increase of 1 unit in body weight, the response variable (i.e., *heart.weight*) will increase by 4.03".

3.2 P-values

Let's look at the p-values of the estimated coefficients (**R**-code is omitted).

```
[1] "Coefficients:"
[2] "            Estimate Std. Error t value Pr(>|t|)    "
[3] "(Intercept)   -0.357      0.692   -0.52     0.61    "
[4] "Body.weight    4.034      0.250   16.12   <2e-16 ***"
```

The column `Pr(>|t|)` in the summary output reports the p-values for each parameter in the model. The predictor *Body.weight* seems to have a very strong effect on the response variable. Indeed, the p-value is very small[2].

Remember that p-values refer to the hypothesis that a given parameter equals zero. In this case, there is thus strong evidence that the slope for body weight is not flat (i.e., it is not zero).

Note that any linear model should include an intercept (see the exercises for more on this topic). Therefore, the p-value associated with the `(Intercept)` parameter is not of interest.

Finally, note that in the very last column of the summary output, **R** assigns stars to highlight significant p-values. Remember that dichotomising p-values into significant/non-significant is very bad practice. Indeed, a p-value of 0.049 is virtually the same as 0.051. In both cases, we conclude that the variable involved seems to have an effect on the response variable. Unfortunately, in scientific publications it has become a widespread rule to dichotomise p-values.

3.3 Including the sex predictor

Let's include the *Sex* predictor into the model[3].

```
lm.cats.2 <- lm(Heart.weight ~ Body.weight + Sex, data = d.cats)
```

Question: *how would you describe this model to a layman. Can you produce a drawing that illustrates this model? (2 minutes to discuss this with your colleagues)*

Let's look at the summary of this model.

```
summary(lm.cats.2)
```

```
Call:
lm(formula = Heart.weight ~ Body.weight + Sex, data = d.cats)

Residuals:
   Min     1Q Median     3Q    Max
-3.583 -0.970 -0.095  1.043  5.102
```

[1] Note that the fact that the intercept does not have a meaningful interpretation does not invalidate the whole model.

[2] **R** uses the scientific notation to report small numbers: thus "2e-16" means 2 times 10 to the power of -16 (basically zero). This is the smallest number the system can show.

[3] Note that here we include the predictors in two steps for didactic reason. In practice, models are fitted with all predictors in one go.

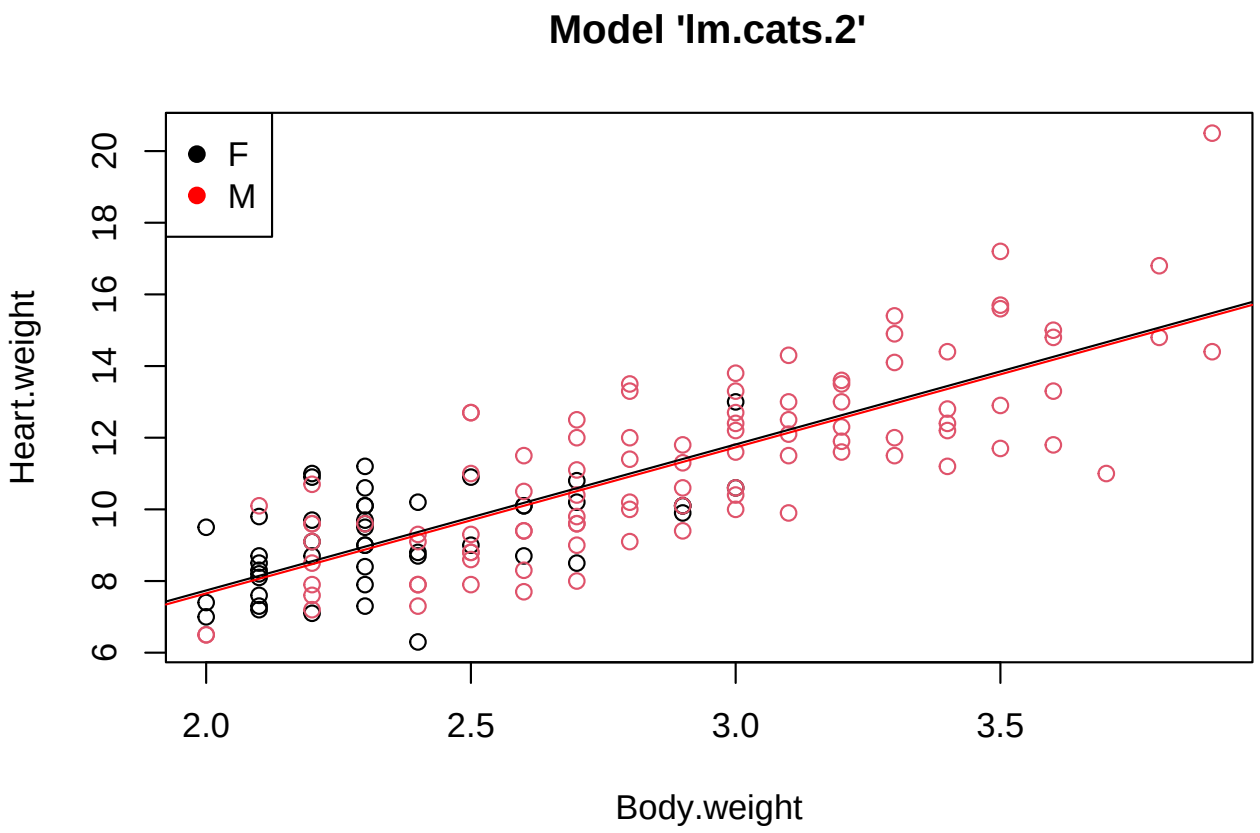
```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  -0.4150     0.7273   -0.57     0.57
Body.weight   4.0758     0.2948   13.83   <2e-16 ***
SexM         -0.0821     0.3040   -0.27     0.79
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.5 on 141 degrees of freedom
Multiple R-squared:  0.647, Adjusted R-squared:  0.642
F-statistic:  129 on 2 and 141 DF,  p-value: <2e-16
```

Question: *how do you interpret the coefficients now?*

The model we just fitted contains an effect for body weight (a slope) and an effect for cat sex (two different intercepts). The natural way to parametrise this model would be to have an intercept for males and an intercept for females. However, **this is not the way linear models are parametrised in** R**!**

Question: *what does the parameter* SexM *refers to? (2 minutes to discuss this with your colleagues)*

Let's visualise the model fit and the data to better understand the **R**-output (for simplicity the **R**-code is omitted).



This graph shows quite clearly that the intercepts of the two groups are essentially the same. The red line (i.e., males) runs slightly below the black line.

With this information in mind is it easier to look at the estimated coefficients and guess the meaning of the `SexM` parameter.

```
coef(lm.cats.2)
```

```
(Intercept) Body.weight        SexM
     -0.415       4.076      -0.082
```

The parameter `SexM` does not represent the intercept for males, as we may have believed, but rather the difference in intercept to females. On the other hand, the `(Intercept)` parameter represents the intercept for females. In other words, the males intercept is the sum of `(Intercept)` and `SexM`. In **R**:

```
## intercept females:
coef(lm.cats.2)["(Intercept)"]
```

```
(Intercept)
      -0.41
```

```
##
## intercept males:
coef(lm.cats.2)["(Intercept)"] + coef(lm.cats.2)["SexM"]
```

```
(Intercept)
       -0.5
```

R deals with categorical variables, which are called "factors" in **R**, by setting one level as being the reference and the other levels as being the difference to that reference. This way of dealing with factors is called "treatment contrasts". Note that the reference level is chosen according to the level names. **R** uses alpha-numerical ordering. So, in this case the level name for females (i.e., F) comes before the one for males (i.e., M) in the alphabet and is thus used as a reference.

See the **Appendix** for more information on the "treatment contrasts" and their uses. Historically, the term "treatment contrasts" comes from clinical studies where different treatments are compared. In this context, it makes sense to have one treatment (the "gold-standard") as a reference.

Going back to the summary for the *lm.cats.2* model, we may want to draw conclusions about the effect of the sex predictor.

```
[1] "            Estimate Std. Error t value Pr(>|t|)    "
[2] "(Intercept)  -0.4150     0.7273   -0.57     0.57    "
[3] "Body.weight   4.0758     0.2948   13.83   <2e-16 ***"
[4] "SexM         -0.0821     0.3040   -0.27     0.79    "
```

The p-value for `SexM` is fairly large (i.e., 0.79). This implies that we cannot exclude that the difference in intercepts between males and females is zero. In other words, there is no evidence for *Sex* to have an effect in this model.

On the other hand, body weight seems to have a very strong effect on the response variable (as seen in the graphical analysis). Indeed, its p-value is very small. In this case, we can easily reject the null hypothesis stating that body weight has no effect ($H_0 : \widehat{\beta_{Body.weight}} = 0$).

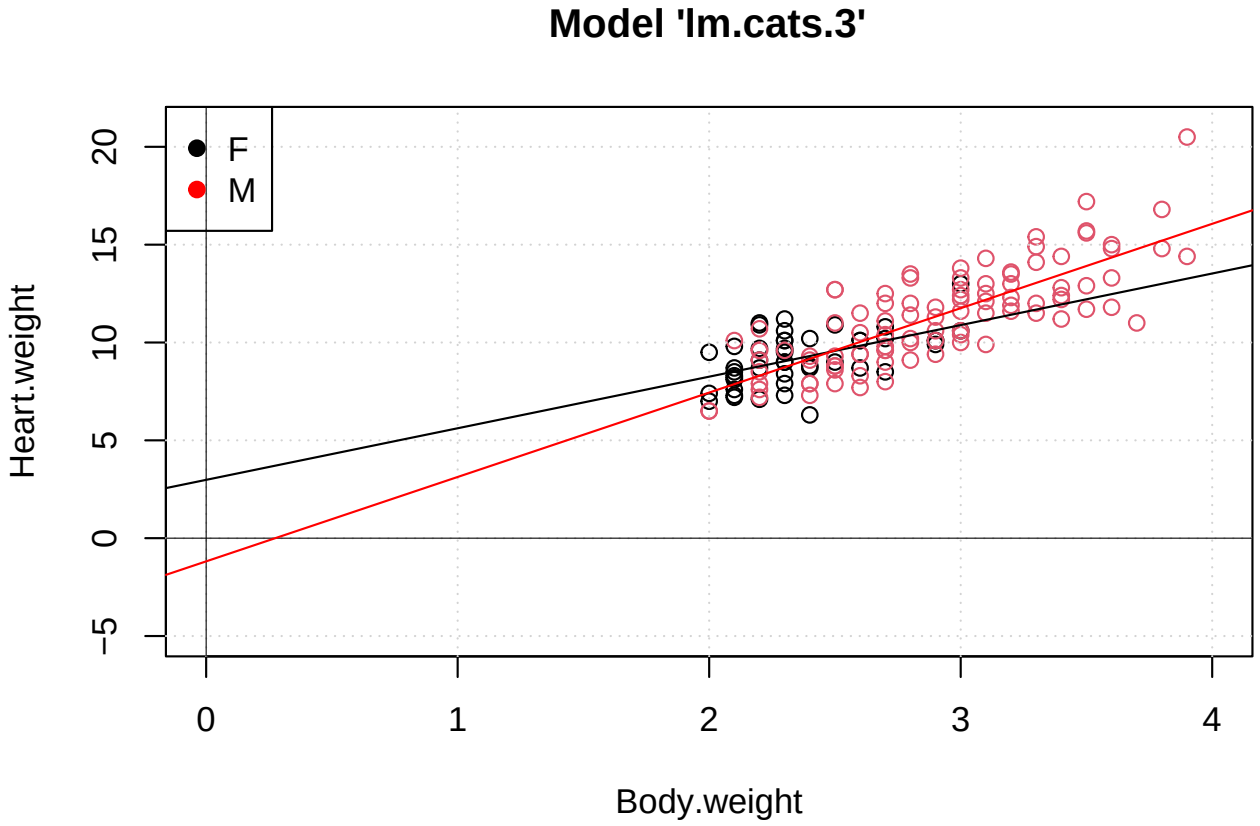
3.4 Including the "sex-body weight" interaction

You may remember from the graphical analysis that males seemed to have a steeper slope for body weight when compared to females. The model we just discussed assumes that the two sexes differ in intercepts only. Let's relax this assumption and fit a model that allows the two sexes to have different intercepts and different slopes[4].

[4] To declare an interaction in **R**-formulae we use the "*" symbol.

```r
lm.cats.3 <- lm(Heart.weight ~ Body.weight * Sex, data = d.cats)
```

Before looking at the model summary, we visualise the model fit along with data. Note that we set the axes limits such that we can better interpret the estimated intercepts. Again, for simplicity, **R**-code is omitted.



Question: *by looking at this graph, can you guess what the value of the intercept for females, respectively for males is? Keep in mind that the treatment contrasts are being used. (2 minutes to discuss this with your colleagues)*

Let's now look at the estimated parameters of this model.

```
coef(lm.cats.3)
```

```
     (Intercept)      Body.weight             SexM Body.weight:SexM
            2.98             2.64            -4.17             1.68
```

As expected, this model has four parameters. The first two parameters refer to the intercept and slope of females. `SexM` is again the difference in the intercept between the two sexes, and `Body.weight:SexM` is the difference in the slopes. We can thus confirm that males seem to have a steeper slope. Indeed, their slope is $2.64 + 1.68 = 4.31$

Side note on **R**-formulae: to declare the formula with an interaction we used the `*` symbol. Equivalently, we could use

```
lm.cats.3.bis <- lm(Heart.weight ~ Body.weight + Sex + Body.weight:Sex, data = d.cats)
```

lm.cats.3 and *lm.cats.3.bis* are fully equivalent models.

Finally, we may want to look at the p-values for these four parameters.

```
summary(lm.cats.3)$coefficients
```

```
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)           3.0       1.84     1.6  0.10796
Body.weight           2.6       0.78     3.4  0.00088
SexM                 -4.2       2.06    -2.0  0.04526
Body.weight:SexM      1.7       0.84     2.0  0.04722
```

There is strong evidence that females have a slope that differs from zero (p-value for *Body.weight* is < 0.001). There is evidence that males have a slope that differs from the females' one (p-value for `Body.weight:SexM` is 0.047).

As discussed above, the interpretation of the intercepts is not biologically relevant here. See "Series 1" for a further discussion on the interpretation of the intercepts and an alternative, more meaningful, parametrisation of this model.

Remember that here, we are only looking at the p-values associated with each single parameter present in the model. This enables us only to draw conclusions about these parameters. In particular, the p-value for:

- `(Intercept)` tests whether the intercept for females is zero
- `Btw` tests whether the slope for females (again the reference group) is different from zero
- `SexM` tests whether the intercept for males differs from the one for females
- `Btw:SexM` tests whether the slope for males differs from the one for females

Next week, we will try to answer more general questions such as "does sex play a role?" or "does a significant interaction imply that both variable play a role?".

3.5 Notes on p-values

Note that we did use the term "significant" or "non-significant" p-values. Possibly the least meaningful way to treat p-values is to dichotomise them into these two latter groups using the 5% threshold (Wasserstein 2016). We prefer to talk about the "amount of evidence against the null hypothesis of a parameter being zero". We therefore used wordings such as "no/weak/some/strong/very strong ... evidence against the null hypothesis of no effect".

Finally, keep also in mind that the p-values presented here rely on several assumptions (e.g., normality of the errors). We will turn our attention to the model assumptions and how these are assessed later in this course.

3.6 "P-values – Confidence intervals" duality

Sometimes (often?) we are not interested in testing an effect (e.g., "does *Body.weight* plays a role or not?"), but rather at estimating an effect and its uncertainty. For example, with the `Cats` data sets we may already know from the literature that there is a positive relationship between heart weight and body weight. In this case, we may be interested in estimating the strength of this relationship. This information may then be useful to dose heart drug precisely.

```
## regression coefficients
coef(lm.cats.3)
```

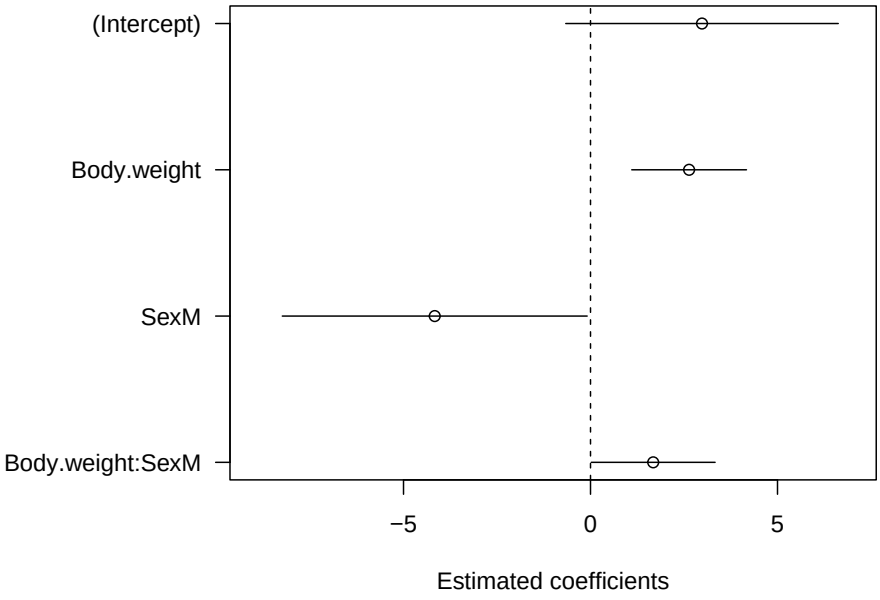
```
     (Intercept)      Body.weight             SexM Body.weight:SexM
             3.0              2.6             -4.2              1.7
```

```
##
## confidence intervals
confint(lm.cats.3)
```

```
                 2.5 % 97.5 %
(Intercept)      -0.662  6.625
Body.weight       1.102  4.170
SexM             -8.242 -0.089
Body.weight:SexM  0.021  3.332
```

The *confint()* function returns, by default, 95% confidence intervals. Note that there is a duality relationship between CI and p-values: a non-significant p-value corresponds to a CI that contains zero. For example, the p-values for `SexM` and `Body.weight:SexM` are very close to significance (4.5% and 4.7% resp.) and indeed they both almost contain zero.

Let's show the estimates along with the confidence intervals (code not shown).



4 Coding conventions

Coding conventions used here:

- data sets are named with the prefix "d." (e.g., `d.cats`).
- linear model objects are named with the prefix "lm." (e.g., `lm.cats`)
- within the text **R**-objects are written using `code formatting` (e.g., `lm.cats`)

- variable names are written using *italic* (e.g., *Body.weight*)
- these conventions are arbitrary, but hopefully useful to the reader

5 Appendix

5.1 Measures of fit

After fitting a model, we may want to be able to comment on the goodness of the fit. A possible way to quantify the goodness of fit is to look at the coefficient of determination R^2. This quantity represents the part of the variation present in the data that is explained by our model. A model that does not explain anything would have an R^2 of zero. On the other hand, a model that explains all the variability of the data would have an R^2 of one.

Let's see how large this value is for our two models.

```
## model with no interaction
formula(lm.cats.2)
```

```
Heart.weight ~ Body.weight + Sex
```

```
summary(lm.cats.2)$r.squared
```

```
[1] 0.65
```

```
##
## model with interaction
formula(lm.cats.3)
```

```
Heart.weight ~ Body.weight * Sex
```

```
summary(lm.cats.3)$r.squared
```

```
[1] 0.66
```

The model with the interaction between sex and body weight has a slightly better R^2. We must note, however, that we are comparing two models that differ in complexity. A model that contains additional parameters will always have a higher R^2. To be able to compare, on a fair basis, two models that differ in their complexity, we can use the *adjusted R^2*. This extension of the coefficient of determination takes into account the number of parameters included in the model.

Let's look at the *adjusted R^2* for these two models.

```
summary(lm.cats.2)$adj.r.squared
```

```
[1] 0.64
```

```
summary(lm.cats.3)$adj.r.squared
```

```
[1] 0.65
```

The model with the interaction still has a slightly higher R^2 after taking into account the number of parameters. In this specific case, R^2 and its adjusted version are pretty much the same (as the two models contain few parameters). Nevertheless, in models with very many parameters, the difference between R^2 and its adjusted version can be large.

Note that the R^2 cannot be used to formally compare two models. In other words, you cannot test and get a p-value on whether a model is better than another using the coefficient of determination. Two linear models are usually compared via F-tests (i.e., Anova tests). We will turn our attention to this topic later in the lecture.

A question that could raise naturally when looking at the coefficient of determination is: "is this a good model fit or not? Which R^2 value indicates a good model fit?". Unfortunately, there is no general threshold that indicates whether a model is good or not. The interpretation of R^2 values greatly changes from case to case.

For example, a model trained on financial data that has a 2% R^2 could make you rich. On the other hand, a model with an R^2 of 40% that was trained on data from a very controlled mechanical experiment may be considered to be a poor model.

5.2 Fitted values

In a few words, the fitted values are the values that a model predicts for those observations that were used to fit the model itself. Fitted values, as well as any other predictions based on a linear model, are computed using the estimated regression coefficients.

For simplicity, here we use the very simple model that only contains *Body.weight* as predictor (i.e., *Sex* is not considered). In this case the fitted values are:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 \cdot Body.weight$$

In **R** we can extract the fitted values of a model by using the function `fitted()`.

```
fitted.cats <- fitted(lm.cats)
##
str(fitted.cats)
```

```
 Named num [1:144] 7.71 7.71 7.71 8.11 8.11 ...
 - attr(*, "names")= chr [1:144] "1" "2" "3" "4" ...
```

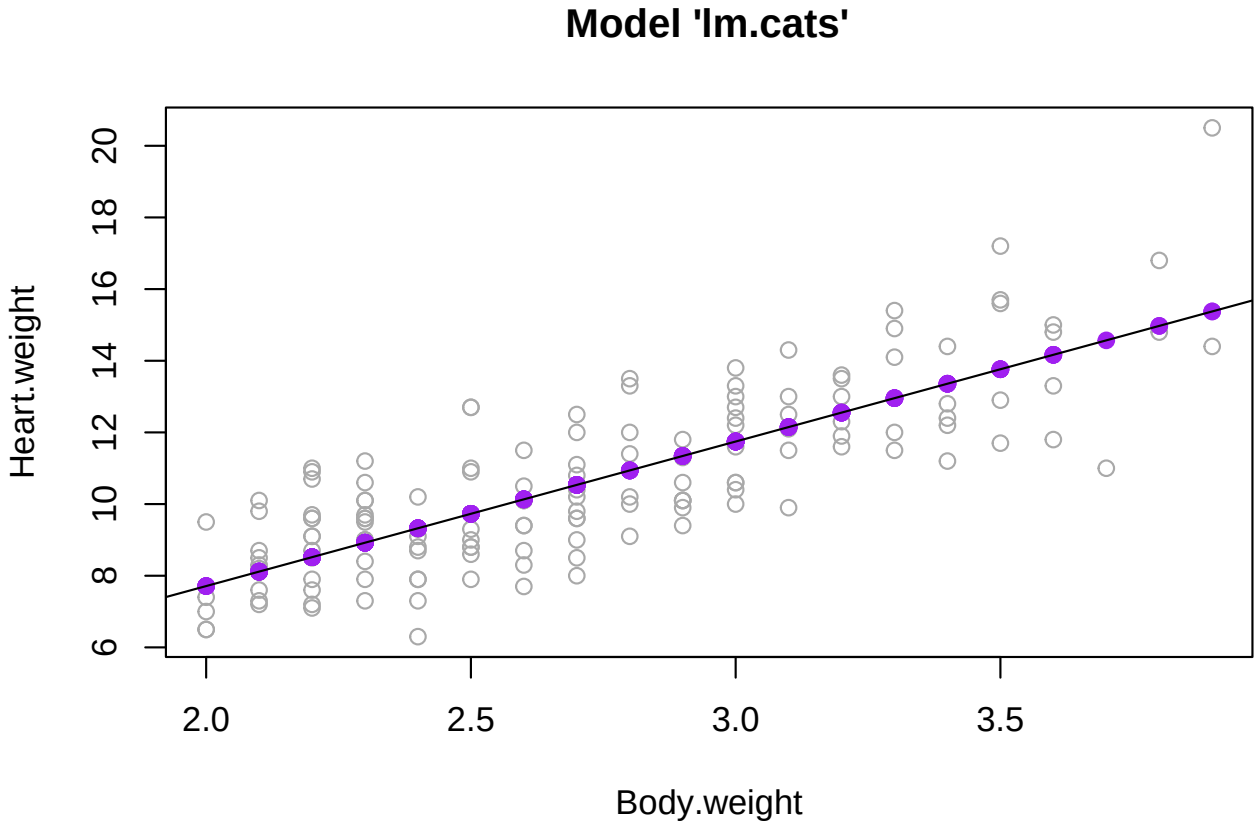
```
head(fitted.cats)
```

```
  1   2   3   4   5   6
7.7 7.7 7.7 8.1 8.1 8.1
```

As expected, the vector of fitted values has length 144. Let's plot these values along with the observed data and the fitted regression line[5].

```
plot(Heart.weight ~ Body.weight, data = d.cats,
     main = "Model 'lm.cats'",
     col = "darkgray")
##
points(fitted.cats ~ Body.weight,
       col = "purple",
       pch = 19,
       data = d.cats)
##
abline(lm.cats, col = "black")
```

[5]Note that for simple regression models with just one predictor, like the one here, we can use the `abline()` function to add the fit to an existing plot.



On this graph we added 144 fitted values. Note, however, that most of them are overlapping. Indeed, the predictor *Body.weight* has only 20 unique values.

5.3 Residuals

Residuals are defined as the difference between the observed values and the fitted values. Let's extract the residuals from the linear model.

```
resid.cats <- resid(lm.cats)
##
length(resid.cats)
```

```
[1] 144
```

```
head(resid.cats)
```

```
    1     2     3     4     5     6
-0.71 -0.31  1.79 -0.91 -0.81 -0.51
```

Let's visualise the residuals along with the data and the model fit. Note for simplicity we only visualise five, randomly selected, residuals.

```
set.seed(20) ## for reproducibility
id <- sample(x = 1:144, size = 5)
resid.cats[id]
```

```
   107    120    130     98     29
 1.554  1.048  1.041 -0.042  1.678
```

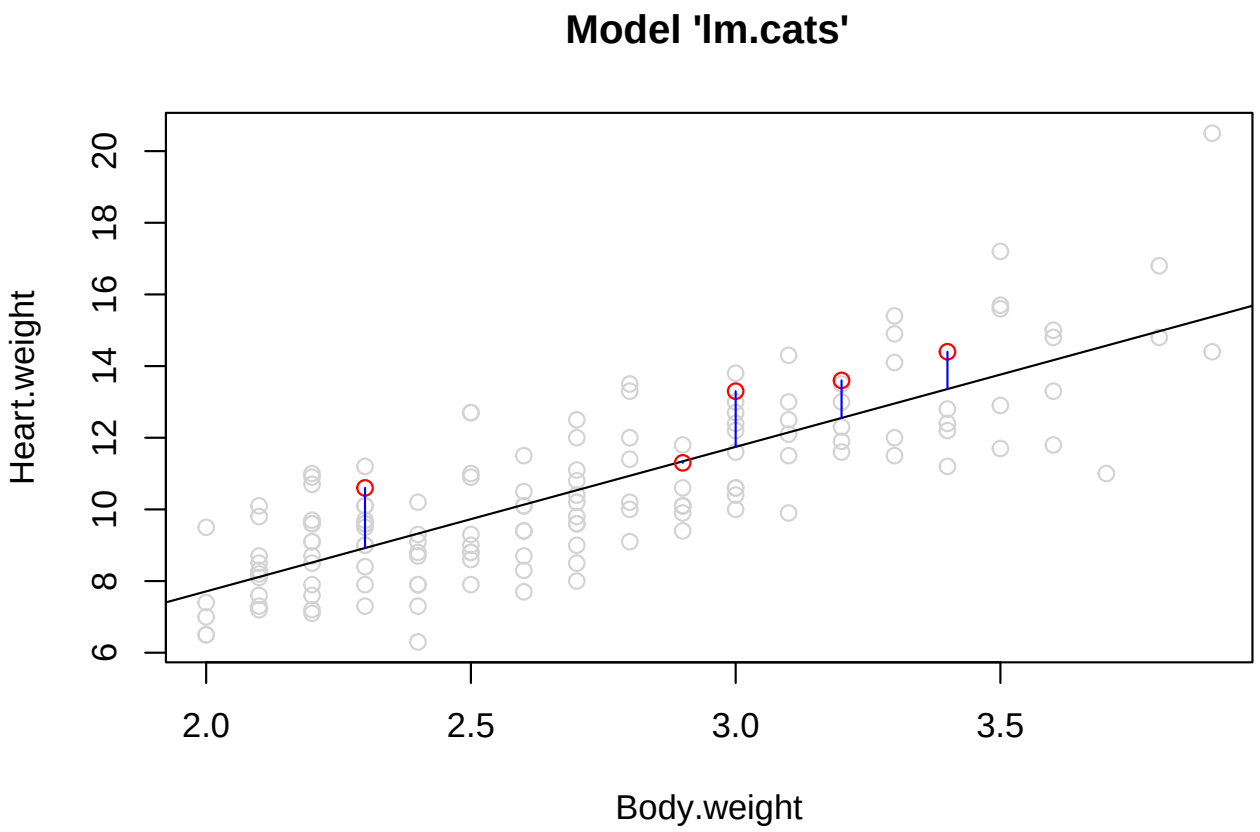
```
fitted.cats[id]
```

```
 107  120  130   98   29
```

```
11.7 12.6 13.4 11.3  8.9
```

Let's visualise these 5 residuals in blue.

```
plot(Heart.weight ~ Body.weight, data = d.cats,
     main = "Model 'lm.cats'",
     col = "lightgray")
##
abline(lm.cats)
##
points(Heart.weight ~ Body.weight, data = d.cats[id, ], col = "red")
##
segments(x0 = d.cats[id, "Body.weight"], x1 = d.cats[id, "Body.weight"],
         y0 = fitted.cats[id], y1 = d.cats[id, "Heart.weight"],
         col = "blue")
```

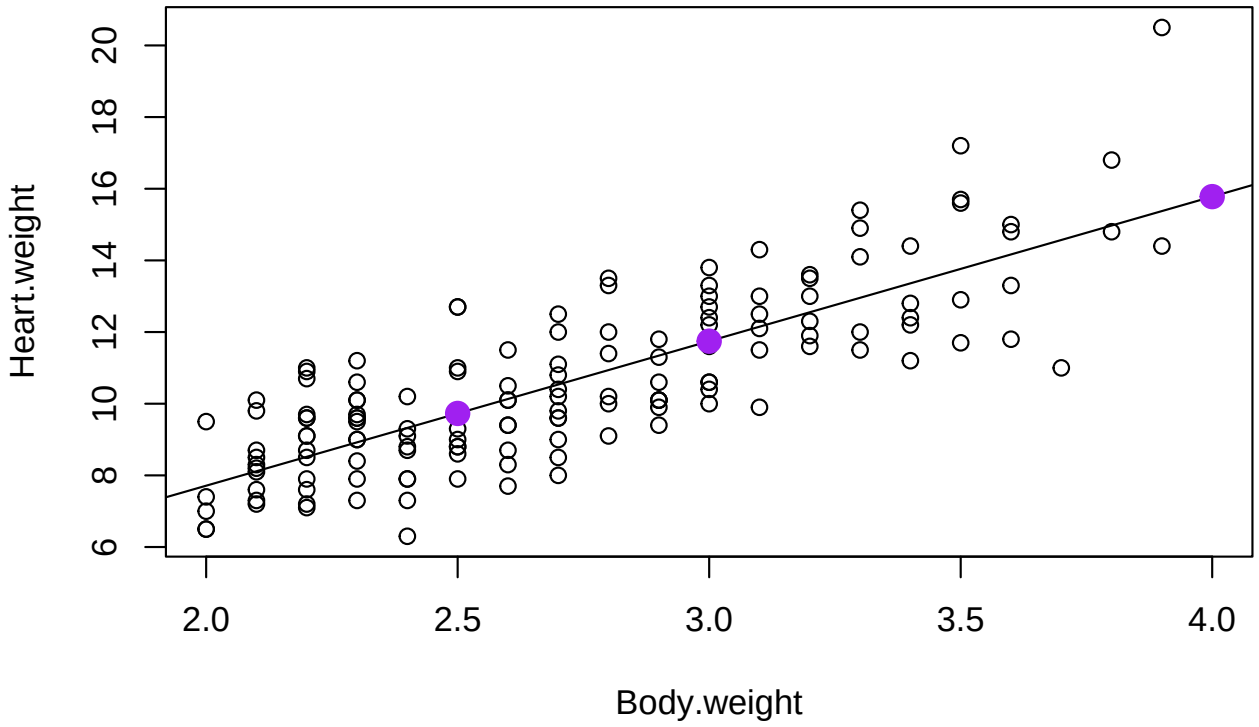


5.4 Predicted values

We have seen above that the function `fitted()` can be used to extract the predicted values for the existing observations. Given a fitted model, we may want to compute predictions for new data. In this case, for example, we may want to make predictions for new cats. For simplicity, we use the model with one predictor only (i.e., *Body.weight*).

```
## 1) create the new data
new.data.cats <- data.frame(Body.weight = c(4, 2.5, 3))
##
## 2) make predictions
pred.new.cats <- predict(object = lm.cats, newdata = new.data.cats)
##
```

```
## 3) display predictions
plot(Heart.weight ~ Body.weight,
     data = d.cats,
     xlim = c(2, 4))
abline(lm.cats)
##
points(x = new.data.cats$Body.weight,
       y = pred.new.cats,
       col = "purple",
       pch = 19, cex = 1.5)
```



It is also possible to compute confidence intervals for the predicted values.

```
pred.new.cats.ci <- predict(object = lm.cats,
                            interval = "prediction",
        newdata = new.data.cats)
pred.new.cats.ci
```

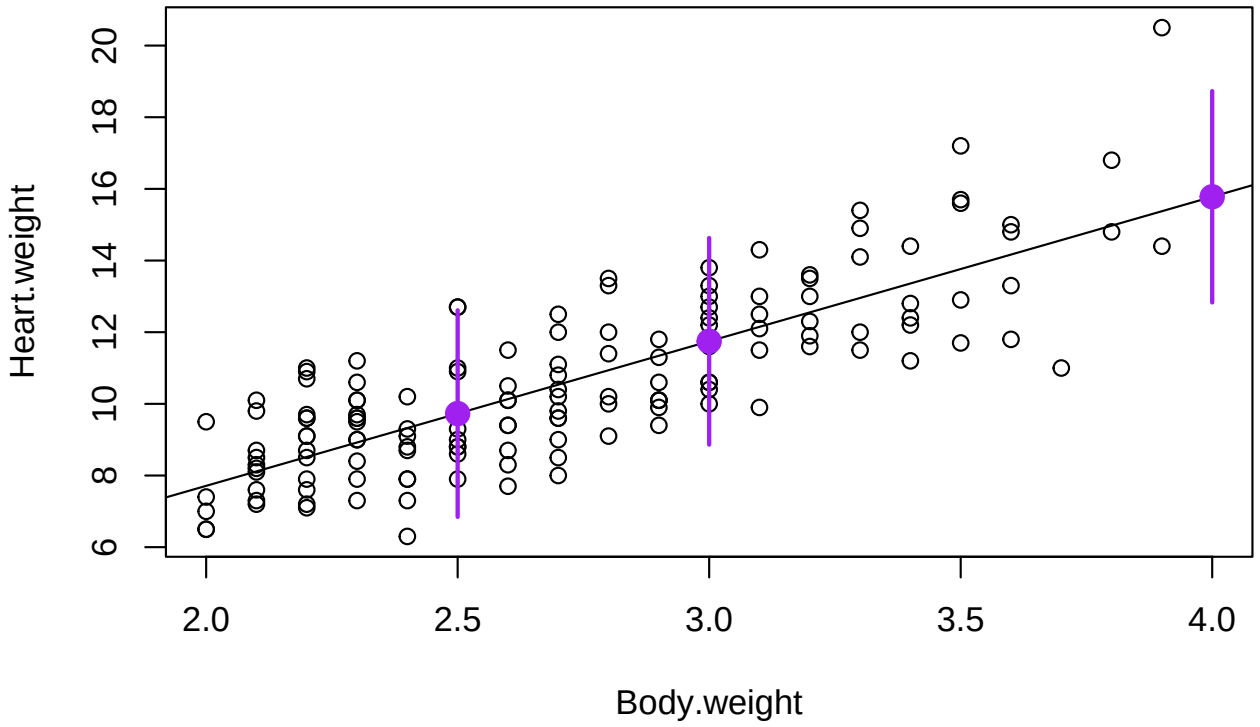
```
   fit  lwr upr
1 15.8 12.8  19
2  9.7  6.8  13
3 11.7  8.9  15
```

These predictions come with a lower (*lwr*) and an upper (*upr*) boundary of the confidence interval (the default confidence level is 95%).

We add this information to the existing graph.

```
plot(Heart.weight ~ Body.weight,
     data = d.cats,
     xlim = c(2, 4))
abline(lm.cats)
##
```

```
points(x = new.data.cats$Body.weight,
       y = pred.new.cats.ci[, "fit"],
       col = "purple",
       pch = 19, cex = 1.5)
##
segments(x0 = new.data.cats$Body.weight,
         x1 = new.data.cats$Body.weight,
         y0 = pred.new.cats.ci[, "lwr"],
         y1 = pred.new.cats.ci[, "upr"],
         lwd = 2,
         col = "purple")
```



5.5 Treatment contrasts

5.5.1 Factors with more than two levels

Categorical variables that have more than two levels are treated the same way as dummy variables (i.e., categorical variables with two levels only, such as *Sex* in the `Cats` dataset). Here, we simulate the situation with three levels. To do that, we set the first 10 observations to the new sex level "Unknown".

```
## 1) add the new level first
levels(d.cats$Sex)
```

```
[1] "F" "M"
```

```
##
levels(d.cats$Sex) <- c("F", "M", "Unknown")
##
levels(d.cats$Sex)
```

```
[1] "F"       "M"       "Unknown"
```

```
##
## 2) set the first 10 observations to unknown
d.cats$Sex[1:10] <- "Unknown"
##
## 3) fit a simple model
lm.cats.Newsex <- lm(Heart.weight ~ Sex, data = d.cats)
##
## 4) look at the coefficients
coef(lm.cats.Newsex)
```

```
(Intercept)        SexM  SexUnknown
        9.6         1.8        -1.6
```

As expected, the level "Unknown" comes after "M". Its estimated regression coefficient represents the difference from the reference level "F".

5.5.2 Other contrasts

Treatment contrasts are the default choice in **R**. These specific type of contrasts make sense for example in clinical settings where we want to compare new drugs with the gold-standard drug. In these settings, it make sense to have the gold-standard drug as a reference and to be able to interpret the effect of the other drugs as the difference to the reference. In other words, we can say how much better is the newer drug working compare to the "old" one.

Note that the type of contrasts used can be changed in **R** via the `options()` function. This is virtually never needed.

5.6 Changing the reference level

It can happen that changing the reference would be required. For example, we may want to set "M" as the reference level.

```
## 1) change reference level
levels(d.cats$Sex)
```

```
[1] "F"       "M"       "Unknown"
```

```
##
d.cats$Sex <- relevel(d.cats$Sex, ref = "M")
##
levels(d.cats$Sex)
```

```
[1] "M"       "F"       "Unknown"
```

```
##
## 2) refit model
lm.cats.relevelled <- lm(Heart.weight ~ Sex, data = d.cats)
coef(lm.cats.relevelled)
```

```
(Intercept)        SexF  SexUnknown
       11.3        -1.8        -3.4
```

As expected, the reference level is now "M".

5.7 How are regression coefficients estimated? (***)

Some of you may be like St. Thomas[6]

[6] According to the legend "he doesn't believe it until he puts his nose into it".

So, you may want to understand how the `lm()` machinery estimates the regression coefficients. To explain this we have to introduce the design (or model) matrix. This latter is the way data is expressed in order to make computations and estimate coefficients.

Let's take a very simple model first. The model that only contains an intercept and the continuous variable *Body.weight*.

```
formula(lm.cats)
```

```
Heart.weight ~ Body.weight
```

```
coef(lm.cats)
```

```
(Intercept) Body.weight
      -0.36        4.03
```

We can write this using mathematical notation.

$$Heart.weight = \beta_0 + \beta_{Body.weight} \cdot Body.weight + \varepsilon$$

More generally, we refer to the response as y and the predictors as x1, x2, ...

$$y = \beta_0 + \beta_1 \cdot x_1 + \varepsilon$$

This can be rewritten in the compact form

$$y = X\beta + \varepsilon$$

Where β is a vector of coefficients (here β_0 and β_1) and X is the design matrix. Let's extract the design matrix from the model to better understand what it is.

```
X <- model.matrix(lm.cats)
dim(X)
```

```
[1] 144   2
```

```
head(X)
```

```
  (Intercept) Body.weight
1           1         2.0
2           1         2.0
3           1         2.0
4           1         2.1
5           1         2.1
6           1         2.1
```

In this case the model matrix has two columns, as we indeed estimate two parameters (an intercept and a slope). The model matrix, which is often denoted as X, has as many rows as the number of observations that were used to fit the model (i.e., 144 observations).

The first few observations shown with `head()` make it clear that the first column of X, which is associated with the intercept, contains only ones. The second one contains the actual values for *Body.weight*.

But how is the design matrix used to get the regression coefficients? The regression estimates are obtained via a linear combination of the design matrix and the observed values (often denoted as y).

$$\widehat{\beta} = (X^T X)^{-1} X^T y$$

let's do this for St. Thomas in **R** [7].

```
y <- model.frame(lm.cats)$Heart.weight
betas.lm.cats <- solve(t(X) %*% X) %*% t(X) %*% y
betas.lm.cats
```

```
             [,1]
(Intercept) -0.36
Body.weight  4.03
```

Excellent! We got the same coefficients as the ones reported by the `coef()` function[8].

Literature

- Lattice: Multivariate Data Visualization with R (Sarkar 2008)
- The ASA Statement on p-Values: Context, Process, and Purpose (Wasserstein 2016)

Session information

```
sessionInfo()
```

```
R version 4.5.1 (2025-06-13)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.1

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK vers

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Europe/Zurich
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
[1] magrittr_2.0.3 ggplot2_3.5.2  knitr_1.50

loaded via a namespace (and not attached):
 [1] vctrs_0.6.5        nlme_3.1-168       cli_3.6.5          rlang_1.1.6
 [5] xfun_0.52          generics_0.1.4     labeling_0.4.3     glue_1.8.0
 [9] htmltools_0.5.8.1  tinytex_0.57       scales_1.4.0       rmarkdown_2.29
[13] grid_4.5.1         evaluate_1.0.4     tibble_3.3.0       fastmap_1.2.0
[17] yaml_2.3.10        lifecycle_1.0.4    compiler_4.5.1     dplyr_1.1.4
[21] RColorBrewer_1.1-3 pkgconfig_2.0.3    mgcv_1.9-3         rstudioapi_0.17.1
[25] lattice_0.22-7     farver_2.1.2       digest_0.6.37      R6_2.6.1
[29] tidyselect_1.2.1   splines_4.5.1      pillar_1.10.2      Matrix_1.7-3
[33] withr_3.0.2        tools_4.5.1        gtable_0.3.6
```

[7] Note the matrix multiplication via %*%.
[8] Note that the `lm()` machinery used a slightly different way to get the inverse of a matrix (i.e., the QR decomposition).